Escalation Dominates Top-Down Allocation Under Bounded Context

Patrick D. McCarthy

Abstract

Top-down allocation — where a supervisory process directs subproblems to lower levels — is viable when the action repertoire $|A|$ fits within bounded active context C_n. We prove that once $|A|$ exceeds a crossover point $|A|^* = C_n/2\alpha$, top-down allocation degrades inference quality monotonically, whether the architecture is flat ($O(|A|)$ overhead) or hierarchical ($O(b \cdot \log_b |A|)$ overhead for branching factor b). The crossover is permanent: recovery requires growing C_n with $|A|$, which no bounded system can satisfy. Escalation — bottom-up failure signaling where higher levels engage only when lower levels fail — achieves $O(1)$ steady-state context cost and *improves* with capability growth, as successfully encoded actions relieve rather than burden the highest level. Since intelligence requires $|A| \gg C_n$, every intelligent system operates beyond the crossover. Escalation dominance is not a universal claim but a scaling result: top-down wins early, escalation wins at every scale that matters. This is a formal critique of precision allocation models including the Free Energy Principle [7, 8] and the System M routing architecture of Dupoux, LeCun, and Malik [6].

1 Introduction

Every system that acts under uncertainty must solve a control problem: which problems receive active reasoning, and which are handled by automatic, pre-encoded responses? This question is distinct from the content of reasoning; it concerns the *architecture of attention*. Under bounded active context C_n — a hard limit on the number of propositions simultaneously available for inference — the control architecture is not a free parameter but a survival-critical design choice.

Two competing architectures present themselves:

1. **Top-down allocation.** A supervisory process at the highest encoding level L_n monitors all subsystems, evaluates relevance, and routes problems to appropriate levels. This architecture is implicit in precision weighting under the Free Energy Principle [7, 8], where higher cortical levels set the gain on prediction errors at lower levels. It is explicit in the System M proposal of Dupoux, LeCun, and Malik [6], where a meta-controller directs when Systems A (observation) and B (action) engage. Kahneman's popular account of System 1 and System 2 [10] implicitly assumes top-down control: System 2 "decides" when to override System 1.

2. **Escalation.** Lower levels handle problems autonomously. Higher levels engage only when lower levels fail — that is, when uncertainty cannot be bounded at the current level. This architecture is implicit in Global Workspace Theory [1, 5], where local processors broadcast to the global workspace only when local resolution fails.

At small scale — when $|A| \leq C_n$, so the entire action repertoire fits in active context — top-down allocation is viable and may even be preferable: the supervisor can hold complete state with room to spare, and the benefit of optimal routing outweighs the modest overhead. Intelligence, however, requires $|A| \gg C_n$: an entity must know far more than it can hold in mind at once. This paper proves that once $|A|$ exceeds C_n, top-down allocation degrades and escalation dominates — and that the crossover is permanent. The result is not that top-down is wrong but that it cannot scale, and scaling is what intelligence demands.

The proof proceeds as follows. Section 2 introduces the encoding hierarchy $E(c, r)$ and the formal definitions required for the result. Section 3 establishes the escalation principle (Theorem 1): informed actions are handled at the lowest sufficient level. Section 4 proves the central result (Theorem 2): top-down allocation degrades inference quality as capability grows, with recovery requiring C_n to grow linearly with capability — a condition no bounded system can satisfy. Section 5 compares the two architectures directly. Section 6 engages with prior work.

This paper is the second in a series. Paper 1 [12] established that any system accumulating knowledge under bounded context necessarily maintains graph structure. The present paper addresses how that structure is *controlled* — how problems are routed through the encoding hierarchy.

2 Formal Framework

We introduce the minimal definitions required for the main results. We inherit the world state space W, proposition, knowledge base K, uncertainty $U(w, K) = -\log P(w \mid K)$ [4], and bounded active context C_n from [12]. An *informed action* $a \in A$ is an action grounded in propositions from K; we write $K^{[a]} \subseteq K$ for the set of propositions that ground a.

Definition 1 (Encoding Hierarchy)**.** An *encoding hierarchy* is a continuous two-dimensional space $E(c, r)$ parameterized by:

- **Runtime cost** $c \geq 0$: the computational cost of accessing and modifying the encoding.
- **Reversibility** $r \in [0, 1]$: the ease with which the encoding can be revised. $r = 1$ is fully reversible; $r = 0$ is permanent.

Boundaries within this continuous space are *interfaces* $L_0, L_1, \ldots, L_n$. The number of interfaces and their positions are substrate-determined: biological systems, artificial systems, and institutional systems partition $E(c, r)$ differently.

Table 1 illustrates the hierarchy with representative interfaces. The precise number and character of levels varies by substrate; what is invariant is the monotonic relationship between runtime cost and reversibility.

Definition 2 (Active Context Capacity and Relevance Density)**.** The *active context capacity* C_n is bounded: $|\mathrm{ctx}(L_n, t)| \leq C_n$. For an informed action a, the *relevance density* at L_n is:

$$\rho(L_n, f) = \frac{|K^{[a]}|}{|\mathrm{ctx}(L_n, t)|}$$

where $K^{[a]} \subseteq K$ is the knowledge relevant to a. $\rho = 1$ is maximum signal: active context holds exactly what a requires. $\rho \to 0$ is noise-dominated: context is filled with content irrelevant to a.

Interface	Description	Runtime Cost	Reversibility
L_n	Active reasoning / working memory	O(compute)	Fully reversible ($r \approx 1$)
L_{n-1}	Learned heuristics / cached policies	$O(1)$ retrieval	Revisable with effort
$\vdots$	*substrate-dependent intermediate levels*		
L_1	Developmental encoding / early learning	Near-zero	Difficult to revise
L_0	Heritable / constitutional encoding	0	Permanent ($r \approx 0$)

Table 1: Representative encoding hierarchy. Cost increases and reversibility increases from L_0 to L_n. The number of interfaces is substrate-determined.

Definition 3 (Delegation Overhead)**.** Under *top-down allocation*, L_n actively manages lower-level processes. For each active delegation, L_n must hold in context:

(i) The *subproblem specification*: what was delegated.

(ii) The *expected return signal*: what a successful result looks like.

(iii) The *integration context*: how the result will be incorporated into the current reasoning state.

Let $\alpha > 0$ be the *minimum context cost per delegation*: the irreducible number of context slots consumed by maintaining one active delegation. $\alpha > 0$ because each delegation requires at least one proposition specifying what was delegated.

3 Escalation

Theorem 1 (Escalation)**.** *Under the encoding hierarchy (Definition 1), the cost-minimizing control policy handles each action a at the lowest level L_i such that the knowledge encoded at L_i is sufficient to bound uncertainty for a. Escalation to L_{i+1} occurs if and only if L_i fails to bound uncertainty.*

Proof. By Definition 1, higher levels have strictly greater runtime cost: $c(L_{i+1}) > c(L_i)$. This monotonicity is not incidental but reflects the architecture of complexity: Simon [14] shows that stable complex systems are *nearly decomposable* — they process information locally, with infrequent communication between subsystems. The encoding hierarchy formalizes this structure: lower levels are cheaper precisely because they are more specialized and less revisable. We show that handling a at any level higher than the lowest sufficient level is strictly dominated.

Let L_i be the lowest level such that $U(w, K_{L_i}^{[a]}) \leq \epsilon$ for the action-relevant threshold ϵ. Consider the alternative of handling a at L_j ($j > i$):

1. **Cost.** $c(L_j) > c(L_i)$ by the monotonicity of cost in the hierarchy. The uncertainty reduction is the same (both levels achieve $U \leq \epsilon$), so the excess cost $c(L_j) - c(L_i) > 0$ yields zero marginal benefit. The higher-level policy is strictly dominated on cost.

2. **Context.** If $j = n$ (the active reasoning level), then handling a at L_n consumes active context capacity C_n. Since C_n is bounded and L_n is the only level that handles genuinely novel problems (problems no lower level can resolve), each unnecessary use of L_n reduces ρ for the next novel problem. The higher-level policy is strictly dominated on context conservation.

Under any cost-minimizing criterion — whether natural selection, engineering optimization, or resource-rational design — the dominated policy is eliminated. The unique cost-minimizing policy is: handle a at the lowest sufficient level; escalate if and only if that level fails.

Remark. This result does not claim that all systems *implement* escalation. Path dependence, architectural constraints, and design choices can produce systems that use higher levels than necessary. The theorem establishes that such systems pay a strictly positive cost premium: they are not cost-optimal under bounded context. The claim is normative (escalation is optimal) rather than descriptive (all systems escalate). □

Corollary 2 (Context Window as Frontier)**.** *L_n is the frontier of unbounded uncertainty, not the seat of intelligence. A problem reaching L_n is evidence of failure at every lower level. An entity that routes most problems through L_n is not "thinking more" — it is failing to encode.*

Corollary 3 (Simultaneous Multi-Level Activity)**.** *All levels of the hierarchy operate concurrently. L_0 handles heritable responses, L_1 handles developmental encodings, and intermediate levels handle their respective domains — simultaneously. L_n handles only the residual: problems that no lower level can resolve. The escalation architecture does not serialize processing through a hierarchy; it parallelizes across levels, with upward propagation occurring only on failure.*

4 Top-Down Allocation Degrades Inference Quality

Theorem 4 (Top-Down Allocation Degrades Inference Quality)**.** *Under top-down allocation, the supervisory level L_n must maintain delegation state for managed subprocesses (Definition 3). Let $k(t)$ be the number of concurrent delegations at time t. The relevance density for a novel problem f_{novel} is:*

$$\rho(L_n, f_{\text{novel}}) = \frac{|K^{[f_{\text{novel}}]}|}{|K^{[f_{\text{novel}}]}| + k(t) \cdot \alpha}$$

If $k(t)$ grows with $|A|$, then $\rho \to 0$ as $|A| \to \infty$. Recovery requires growing C_n with $k(t)$ — a condition no bounded system can satisfy.

Proof. Under top-down allocation, L_n serves two simultaneous functions: (a) reasoning about the novel problem f_{novel}, requiring $|K^{[f_{\text{novel}}]}|$ context slots, and (b) maintaining delegation state for $k(t)$ concurrently managed subprocesses, each consuming α context slots (Definition 3).

The total context consumed is:

$$|\text{ctx}(L_n, t)| = |K^{[f_{\text{novel}}]}| + k(t) \cdot \alpha$$

Since $\alpha > 0$, any growth in $k(t)$ increases the denominator while the numerator $|K^{[f_{\text{novel}}]}|$ is bounded per problem. Therefore ρ decreases monotonically with $k(t)$.

The critical question is whether $k(t)$ grows with $|A|$. We show that it must under any architecture that merits the name "top-down":

Why k grows with $|A|$. Top-down allocation is defined by the supervisory level *initiating* and *tracking* delegations (Definition 3). A supervisor that delegates but does not track (fire-and-forget) holds no return-integration context and cannot evaluate, redirect, or integrate results — it is not supervising. A supervisor that tracks only a fixed subset of $|A|$ while ignoring the rest has abandoned top-down control of the ignored actions, which are then self-managing (i.e., operating under escalation). In either case, the system is no longer top-down for the untracked actions.

For the portion of A that remains under genuine top-down supervision, $k(t)$ grows with the number of supervised actions. Under flat top-down, $k = |A_{\text{supervised}}|$. Under hierarchical top-down with branching factor b, each supervisory level tracks b subordinates across $\lceil \log_b |A_{\text{supervised}}| \rceil$ levels, giving $k = b \cdot \log_b |A_{\text{supervised}}|$. In both cases k grows with the supervised repertoire.

The critical threshold occurs when $k(t) \cdot \alpha \geq C_n - |K^{[f_{\text{novel}}]}|$: delegation overhead has consumed all context capacity not occupied by the current problem's knowledge.

To maintain ρ at a fixed level as $|A|$ grows, C_n must grow at least linearly with $k(t)$. But C_n is bounded by substrate (Definition 2). Therefore top-down allocation necessarily degrades inference quality as the supervised repertoire grows under bounded context. □

4.1 The crossover: when top-down stops working

Top-down allocation is viable when delegation overhead fits comfortably within C_n. Define the *crossover point* as the supervised repertoire size at which delegation overhead consumes half of active context:

$$k(|A|^*) \cdot \alpha = \frac{C_n}{2}$$

For flat top-down ($k = |A|$), this gives $|A|^* = C_n/2\alpha$. For hierarchical top-down ($k = b \cdot \log_b |A|$), the crossover is $b \cdot \log_b |A|^* = C_n/2\alpha$, which gives a larger $|A|^*$ — hierarchy delays the crossover but does not eliminate it, because $\log_b |A|$ still grows without bound.

Below $|A|^*$, the supervisor retains at least $C_n/2$ slots for reasoning — adequate for most problems. Above $|A|^*$, delegation overhead dominates: every new supervised capability *reduces* the system's ability to reason about novel problems.

Estimating the crossover. The minimum delegation cost α is at least 1: specifying "what was delegated" requires at least one proposition in context. A tighter estimate: each delegation requires a specification, an expected return, and an integration slot (Definition 3), giving $\alpha \geq 3$. Under Cowan's [3] revised estimate of working memory capacity ($C_n \approx 4$, not Miller's 7 ± 2), the flat crossover is $|A|^* = C_n/2\alpha \approx 4/6 < 1$. Even with Miller's more generous bound ($C_n \approx 7$), $|A|^* \approx 7/6 \approx 1$. The crossover for biological working memory occurs at *one delegation* — a supervisor that manages even a single subprocess has already consumed most of its reasoning capacity. This is why human "multitasking" degrades performance: the delegation overhead of tracking two tasks consumes the context needed to reason about either.

This crossover is permanent under any top-down architecture. Once $k(|A|) \cdot \alpha > C_n/2$, the only recovery is to grow C_n — which is bounded by substrate — or to abandon top-down supervision for part of A, which is to adopt escalation for those actions.

The crossover clarifies the scope of the result. Top-down allocation is not wrong — it is a *small-scale* architecture. For an entity whose entire repertoire fits in active context, top-down supervision is efficient and may be optimal. But intelligence requires $|A| \gg C_n$: an entity that can only act on what it can simultaneously hold in mind is not intelligent but reactive. Every physical system that we recognize as intelligent — from insects routing foraging with stigmergic signals to humans operating with ∼7 active context slots across thousands of learned skills — operates well beyond the crossover.

Cell division as illustration. A single-celled organism can "manage" its repertoire top-down: the entire state fits in one context. A multicellular organism with 10^{13} cells cannot manage each cell individually through a central controller.

Biology's actual solution is a *hybrid*: local autonomy (cells monitor nutrient gradients, contact inhibition, and growth factor concentrations) combined with systemic signaling (hormonal regulation, tumor suppressors, immune surveillance). The systemic signals are genuine top-down control — but they are *broadcast* signals (growth hormone affects all cells of the right type), not individual delegations (no central process tracks each cell's state). Broadcasting is $O(1)$ in delegation context: the signal is sent once regardless of the number of recipients. Individual delegation would require tracking 10^{13} cells, which is well beyond any bounded context.

The hybrid architecture is precisely the crossover result: top-down broadcast control for system-wide coordination (below the crossover for broadcast), combined with local escalation for individual cell decisions (above the crossover for per-cell management). When the systemic signaling fails — when a cell ignores growth suppression signals — the immune system engages: escalation on failure.

4.2 The hierarchical top-down objection

The strongest objection to Theorem 4 is that it attacks flat top-down allocation — a strawman. Real supervisory systems are *hierarchical*: a top level manages b intermediaries, each managing b sub-intermediaries, so that overhead at any single level is $O(b)$, not $O(|A|)$. Does hierarchical decomposition rescue top-down allocation?

No. The hierarchical decomposition is more subtle than the flat case, but it does not rescue top-down allocation. We address the strongest version of the objection directly.

(1) L_n sees only $O(b)$ overhead — but the hierarchy has a different cost. The strongest version of the hierarchical objection notes that L_n manages only b direct subordinates, so overhead at L_n is $b \cdot \alpha$ — constant in $|A|$. The $\log_b |A|$ levels of overhead are distributed across intermediate levels, each with its own bounded context. No single level overflows. This is correct and is a genuine advantage of hierarchical decomposition over flat top-down.

However, the cost is not eliminated — it is *redistributed*. The hierarchy requires $\lceil \log_b |A| \rceil$ supervisory levels, each of which devotes $b \cdot \alpha$ of its context to delegation state. The number of management-dedicated levels grows with $|A|$.

(2) Management levels displace knowledge levels. This is the deeper cost, and it has a precise analogue in organizational economics. Williamson [15] proves that each management layer in a hierarchical organization introduces "control loss": a fraction $\alpha_W < 1$ of the intended directive is lost at each level, so that n management levels transmit only α_W^n of the top-level intent. This limits the viable depth of hierarchical control and, consequently, organization size. The same structure applies here: each intermediary in a hierarchical top-down system is itself a supervisory process that must hold delegation context, and the cumulative cost of these management layers limits the encoding capacity available for knowledge.

In an escalation architecture, every encoding level is a *knowledge level*: it holds proven actions and propositions, not routing state. In hierarchical top-down, $\lceil \log_b |A| \rceil$ levels are *management levels*: they hold delegation state. As $|A|$ grows, the number of management levels grows (logarithmically), consuming encoding capacity that escalation would use for knowledge. Williamson's result shows this is not merely wasteful but structurally limiting: the management overhead places an upper bound on the effective depth of hierarchical control, just as it places an upper bound on firm size.

The cost is not context overflow at any single level but a growing fraction of the entity's encoding hierarchy devoted to management rather than knowledge. An entity with 10 encoding levels and $|A|$ requiring 4 management levels has only 6 levels for knowledge; escalation would use all 10. The management overhead is in *levels consumed*, not *context slots consumed at L_n*.

(3) The causal direction is the fundamental difference. The deepest distinction between hierarchical top-down and escalation is not the cost asymptotic but the *causal direction of engagement*:

- In hierarchical top-down, the upper level *initiated* the delegation. It is waiting for a result it requested. It must hold return-integration context: what was delegated, what a successful result looks like, and how to incorporate it. This context is held *whether or not the subprocess encounters difficulty.* A supervisor waiting for ten reports holds ten slots regardless of whether any report is late.
- In escalation, the upper level *did not initiate* anything. It holds zero context for problems handled below. It loads context only when a failure signal arrives from below — and in steady state, failures are rare because lower levels handle most problems. There is nothing to "wait for" because nothing was requested.

This is not a quantitative difference in overhead — it is a structural difference in what context is held and why. Hierarchical top-down holds *anticipatory context* (waiting for results that may or may not arrive). Escalation holds *reactive context* (responding to failures that have already occurred). Anticipatory context scales with the number of delegations; reactive context scales with the failure rate. As the system matures and lower levels improve, the failure rate decreases, so escalation overhead *decreases with capability.* In hierarchical top-down, the number of delegations *increases with capability*: more capability means more subprocesses, each requiring a delegation slot.

Remark on convergence. Hierarchical top-down that is sufficiently deep, with intermediaries that handle most problems without escalating, begins to *resemble* escalation. The intermediary that resolves a problem without reporting upward is functioning as an escalation level: it holds no delegation context at the level above. The convergence is one-directional: as hierarchical top-down becomes more autonomous at lower levels, it converges toward escalation. Escalation never converges toward top-down, because it never introduces anticipatory delegation context.

4.3 Event-driven "top-down" and causal direction

A fourth objection: an event-driven top-down supervisor registers callbacks, holds zero delegation state, and is invoked by subsystems on completion or failure. This achieves escalation's $O(1)$ steady-state cost while retaining "top-down authority."

The issue is not one of labeling — it is not that we absorb every efficient architecture under the name "escalation" to make the thesis unfalsifiable. The issue is the *causal direction of the signal.* The structural distinction between top-down and escalation is:

- **Top-down**: the upper level initiates. The signal goes *downward.* The upper level says "do this" and waits for the result.
- **Escalation**: the lower level signals. The signal goes *upward.* The lower level says "I cannot handle this" and the upper level engages reactively.

In the event-driven architecture, who initiates the callback? The *subsystem.* The subsystem detected the condition — whether completion or failure — and sent the signal *upward.* The upper

level did not poll, did not check, did not anticipate. It was invoked from below. This is bottom-up signaling regardless of what the architecture is called.

The claimed top-down authority — the ability to initiate, cancel, or redirect — is unavailable to a supervisor holding zero state. A supervisor with no delegation context does not know what is happening below and cannot redirect what it is not tracking. When the callback arrives, the supervisor must *reload context* to interpret the result — exactly what an escalation level does when a failure signal arrives.

The point is not definitional but empirical: if you trace the causal arrow in the event-driven architecture, it points upward. The architecture may have been *designed* as top-down (the supervisor originally assigned the task), but in steady-state operation, the information flow is bottom-up. The label "top-down" describes the initial assignment; the ongoing control structure is escalation.

4.4 Hidden costs of escalation during learning

A second objection: escalation achieves $O(1)$ steady-state cost, but this assumes lower levels are competent. During learning — when lower levels frequently fail — escalation produces frequent failure signals at L_n, burdening it with escalation context.

This is correct but not fatal to the argument. During learning, both architectures burden L_n: top-down burdened by delegation overhead plus the novel problems themselves; escalation burdened only by the novel problems (the failures that escalate). The difference: under top-down, the overhead is *additive* to the learning load (delegation context plus problem context). Under escalation, the overhead *is* the learning load (failure context only). Escalation during learning is not $O(1)$ — it is $O(\text{failure rate} \cdot |K^{[a]}|)$ — but it never exceeds the intrinsic difficulty of the problems themselves. Top-down adds coordination overhead on top of intrinsic difficulty.

As learning proceeds and lower levels encode proven actions, escalation load decreases monotonically. Top-down delegation overhead does not decrease with learning — it grows with the number of managed processes, which increases as the system becomes more capable.

4.5 Empirical dominance of dense attention

A third objection: transformers with full self-attention — the purest top-down architecture, where every token attends to every other — have dominated alternative architectures for a decade. If top-down truly degraded, why does dense attention win?

The answer is in the trajectory, not the snapshot. Dense attention dominates *today* at *current scales*. But the trend is toward escalation-like structure:

- **Sparse attention** (learned sparsity patterns, local windows, routing) reduces $O(n^2)$ by learning which edges to keep — learned graph pruning [12].
- **Mixture-of-experts** routes tokens to specialized sub-networks rather than evaluating all parameters for all tokens — escalation at the parameter level.
- **Retrieval-augmented generation** retrieves relevant context from an external store rather than holding everything in the context window — graph traversal from a query node.
- **"Lost in the middle"** [11]: empirical evidence that inference quality degrades as context fills with irrelevant content — precisely the ρ degradation of Theorem 4.

The industry is converging toward escalation. Dense attention is the starting point, not the destination. The cost pressures the theorem identifies are driving the convergence.

Remark: the LLM theory of the case. Theorem 4 does not claim that large language models *cannot work*. Dense attention over a large context window is a viable strategy as long as the context can cover the problem space — that is, as long as the system operates below the crossover (Section 4.1). Current LLMs succeed precisely because their context windows are large enough, relative to the problems they are asked to solve, that ρ degradation is tolerable.

What the theorem establishes is that this strategy is *suboptimal and expensive*. Full self-attention recomputes the pairwise adjacency structure from scratch at every query: $O(n^2)$ per inference step. A system with stored adjacency pays $O(d)$ per retrieval, where d is the local degree. The cost ratio grows quadratically with context size. For a 128k-token context window, the system computes $\sim 10^{10}$ pairwise attention weights per layer per query, the vast majority of which are near-zero (irrelevant pairs). This is the computational cost of not having a graph.

The theorem also predicts a ceiling. As the problems LLMs are asked to solve grow in complexity — requiring more knowledge, more reasoning steps, more integration across domains — the context required grows with the problem, and ρ degrades: "lost in the middle" [11] is the empirical manifestation. Growing the context window defers the crossover but does not eliminate it, because the overhead of attending to irrelevant content grows quadratically while the benefit of additional context grows sub-linearly (most added context is irrelevant to any specific problem).

The LLM architecture is not invalidated by this result — it is *located* by it. LLMs operate in the pre-crossover regime where dense top-down attention is functional but costly. The industry trajectory toward sparse attention, mixture-of-experts, and retrieval-augmented generation is movement toward the post-crossover architecture the theorem identifies as necessary: stored adjacency, selective retrieval, escalation on failure.

Remark on scope. Theorem 4 is not a claim that top-down control is wrong in all cases. Occasional top-down intervention — where L_n deliberately overrides a lower-level response — is a valid and sometimes necessary operation. The theorem establishes that top-down allocation as the *primary* control architecture degrades under bounded C_n. The distinction is between top-down as occasional intervention (compatible with escalation) and top-down as the default routing mechanism (incompatible with bounded context).

Biological measurement. The empirical signature of escalation is the transition from active to automatic processing across skill acquisition. When an informed action a is first learned, it consumes L_n capacity: the learner must actively reason through each step. As encoding proceeds — as $K^{[a]}$ migrates from L_n to lower levels — a ceases to consume L_n capacity. This is precisely the escalation architecture: a has been encoded at a level where it no longer requires active context. The ubiquity of this transition across motor learning, language acquisition, and expert skill development [10] is evidence that biological systems implement escalation rather than persistent top-down allocation.

5 Comparison: Escalation vs. Top-Down

The two architectures can be compared directly on their steady-state context demands.

5.1 Escalation architecture

Under escalation, L_n engages only when lower levels fail to bound uncertainty for a given problem.

- **Steady-state context per problem:** $O(|K^{[a]}|)$. Only the knowledge relevant to the currently escalated problem occupies L_n. No delegation overhead is incurred because lower levels are not "managed" — they operate autonomously.
- **Effect of knowledge growth:** As K grows and more actions are encoded at lower levels, *fewer* problems reach L_n. The load on active context decreases or remains bounded as capability increases.
- **Scaling property:** Capability growth *relieves* L_n rather than burdening it. Each successfully encoded action is one fewer demand on active context.

5.2 Top-down allocation architecture

Under top-down allocation, L_n holds delegation context for all managed subprocesses.

- **Steady-state context per problem:** $O(|K^{[a]}| + k \cdot \alpha)$, where k is the number of concurrent delegations. The delegation overhead $k \cdot \alpha$ is additive to the problem's own knowledge requirements.
- **Effect of capability growth:** As A grows, k grows proportionally (the supervisor must track more subprocesses). L_n load increases with capability.
- **Scaling property:** Capability growth *burdens* L_n. Each new capability adds to the supervisory overhead, eventually saturating active context.

5.3 The inversion

The escalation architecture inverts the scaling relationship between capability and cognitive load:

	Escalation	**Top-Down**
L_n engagement	On failure only	Continuous supervision
Steady-state context	$O(\|K^{[a]}\|)$	$O(\|K^{[a]}\| + k \cdot \alpha)$
As $\|A\|$ grows	L_n load $\downarrow$ or bounded	L_n load $\uparrow$ proportionally
At capacity	Novel problems get full C_n	Novel problems get $C_n - k\alpha$

Under escalation, a system that has encoded 10,000 actions at lower levels has *more* active context available for novel problems than when it had encoded 100 — because 9,900 additional problems are now handled without L_n engagement. Under top-down allocation, a system managing 10,000 subprocesses has *less* active context available than when it managed 100 — because 9,900 additional delegation slots consume context.

This is not a quantitative difference but a qualitative inversion of the scaling direction. It explains why expertise feels like *effortlessness*: the expert has encoded actions that the novice must actively reason through, freeing L_n for genuinely novel problems.

6 Engagement with Prior Work

6.1 Free Energy Principle and Active Inference

The Free Energy Principle [7, 8] posits that biological systems minimize variational free energy, with attention implemented as precision weighting: higher cortical levels set the gain on prediction errors at lower levels. This is a top-down architecture in the sense of Theorem 4: the supervisory level must maintain a representation of expected precision for each lower-level channel, and must evaluate incoming prediction errors against these expectations.

Theorem 4 shows that this precision allocation degrades under bounded C_n because the delegation overhead — maintaining expected precision for k channels — fills the active context window proportionally to the number of monitored channels. This is a critique of the architectural *prescription*, not of the minimization objective. Free energy minimization may well be the correct objective function; the question is whether the control architecture that implements it should be top-down (precision allocation from above) or bottom-up (escalation on prediction error that exceeds local capacity). Under bounded context, escalation dominates.

More precisely: FEP's precision weighting may be optimal when C_n is not binding — when active context is large relative to the number of monitored processes. Under bounded context, the overhead of maintaining precision weights for all channels eventually exceeds the benefit of optimal allocation, and escalation (where channels signal only on failure) achieves better inference quality at L_n.

6.2 System M: Dupoux, LeCun, and Malik

Dupoux, LeCun, and Malik [6] propose a three-component architecture: System A (observation/world models), System B (action/planning), and System M (meta-control), where M is a fixed evolutionary routing policy that directs when A and B engage. System M monitors epistemic signals and routes data between subsystems.

This is explicitly top-down: M sits above A and B, monitoring their states and directing their activity. Theorem 4 establishes that M's monitoring itself consumes C_n proportionally to the number of subsystems and channels it oversees. As the architecture scales — more world models, more action policies, more channels — M's overhead grows, eventually crowding out the capacity for the reasoning that M is supposed to coordinate.

Their architecture would benefit from replacing top-down routing with escalation: subsystems A and B signal M only on failure (when local uncertainty exceeds a threshold), rather than M continuously polling all subsystems for state. Under escalation, M engages only when a subsystem cannot resolve its own uncertainty — and the overhead at M's level is proportional to the *failure rate*, not the *system size*.

6.3 Global Workspace Theory

Global Workspace Theory [1, 5] already describes what is essentially an escalation architecture: specialized local processors handle routine computation autonomously, and the global workspace is engaged only when local processing fails or when integration across modules is required. Broadcasting to the global workspace is the cognitive analogue of escalation to L_n.

Theorem 4 provides the formal grounding that GWT assumes architecturally. GWT *asserts* that local processors handle routine work and the global workspace handles exceptions; Theorem 1

derives the escalation criterion from uncertainty-reduction cost, and Theorem 4 *proves* why the alternative (top-down allocation from the global workspace) fails. The $O(k \cdot \alpha)$ overhead is the formal reason why global workspace engagement must be the exception rather than the rule.

6.4 System 1 and System 2

Kahneman's System 1/System 2 framework [10] is the popular binary version of the encoding hierarchy. System 1 (fast, automatic, low-cost) corresponds to lower levels of $E(c, r)$; System 2 (slow, deliberate, high-cost) corresponds to L_n.

The continuous parameterization $E(c, r)$ corrects the binary discretization. There are not two systems but a continuous space of encodings with substrate-determined interfaces. A motor reflex ($c \approx 0$, $r \approx 0$) and a well-practiced heuristic (c low, r moderate) are both "System 1" in Kahneman's taxonomy, but they occupy different regions of $E(c, r)$ with different costs and different revisability. The binary collapses this structure into a distinction that, while useful for popular exposition, obscures the continuous nature of encoding and the multiplicity of interfaces.

Theorem 1 also corrects the implicit control assumption: System 2 does not "decide" when to override System 1. Rather, System 1 (lower levels) handles problems autonomously, and System 2 (L_n) engages when System 1 fails. The direction of control is bottom-up, not top-down.

6.5 Resource Rationality

Griffiths, Lieder, and Goodman [9] establish that the optimal algorithm for a cognitive task depends on the computational resources available — resource rationality. The optimal unconstrained algorithm may be suboptimal under resource limitations.

Theorem 4 is precisely an instance of this principle. Top-down allocation may be the optimal control architecture when C_n is unbounded: with infinite active context, the overhead of maintaining delegation state for all subprocesses is negligible, and the benefit of optimal routing is maximal. Under bounded C_n, the overhead is no longer negligible, and escalation — a "simpler" architecture that does not attempt global optimization of routing — dominates because it conserves the scarce resource (active context) for genuine inference rather than spending it on coordination.

Resource rationality provides the general principle; Theorem 4 provides the specific result for the control architecture domain: the resource-rational control architecture under bounded context is escalation.

7 Conclusion

We have shown that escalation dominance is a *scaling result*. Top-down allocation is viable — and may be optimal — when the action repertoire $|A|$ fits within bounded context C_n. But intelligence requires $|A| \gg C_n$: an entity must know far more than it can hold in mind at once. Once $|A|$ exceeds the crossover point $|A|^* = C_n/2\alpha$ (Section 4.1), top-down delegation overhead dominates active context and inference quality degrades monotonically. The crossover is permanent under bounded C_n.

The argument has three components:

1. Theorem 1 establishes that informed actions are handled at the lowest sufficient level, with escalation occurring only on failure.

2. Theorem 4 establishes that top-down allocation degrades inference quality as $|A|$ grows, whether the architecture is flat ($k = |A|$) or hierarchical ($k = b \cdot \log_b |A|$). Recovery requires unbounded growth of C_n, which contradicts the boundedness assumption.

3. The crossover analysis (Section 4.1) identifies the regime boundary: top-down works below $|A|^*$, fails above it. Every system we recognize as intelligent operates well above it.

The combination yields a precise result: escalation is not universally superior to top-down — it is superior at every scale that matters. L_n is the level of last resort, not the seat of control. Capability growth, under escalation, *relieves* the highest level rather than burdening it. Under top-down allocation, capability growth *burdens* the highest level until it saturates.

This result has implications for both cognitive science and AI architecture. In cognitive science, it provides the formal grounding for Global Workspace Theory's architectural assumptions and corrects the implicit top-down control assumption in System 1/System 2 accounts. In AI architecture, it identifies a structural ceiling on centralized routing: systems scaling beyond $|A|^*$ should implement escalation — subsystems that operate autonomously and signal on failure — rather than supervisory routing that attempts to manage all subprocesses from above. The trajectory of transformer architectures, from dense attention toward sparse attention, mixture-of-experts, and retrieval-augmented generation, is convergence toward this result.

The series to this point has established two results: knowledge under bounded context necessarily forms a graph [12]; and the control architecture over that graph is necessarily escalation under bounded context (this paper). Together, these results constrain the architecture of any system that accumulates knowledge, acts under uncertainty, and operates within bounded active context — which is every physical system that persists.

References

[1] B. J. Baars. *A Cognitive Theory of Consciousness.* Cambridge University Press, 1988.

[2] A. Clark. *Surfing Uncertainty: Prediction, Action, and the Embodied Mind.* Oxford University Press, 2015.

[3] N. Cowan. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1):87–114, 2001.

[4] T. M. Cover and J. A. Thomas. *Elements of Information Theory.* Wiley-Interscience, 2nd edition, 2006.

[5] S. Dehaene. *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts.* Viking Press, 2011.

[6] E. Dupoux, Y. LeCun, and J. Malik. Why AI systems don't learn and what to do about it: Lessons on autonomous learning from cognitive science. *arXiv preprint arXiv:2603.15381*, 2026.

[7] K. Friston. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.

[8] K. Friston. Active inference and learning. *Neuroscience & Biobehavioral Reviews*, 68:862–879, 2017.

[9] T. L. Griffiths, F. Lieder, and N. D. Goodman. Rational use of cognitive resources: Levels of analysis between the computational and the algorithmic. *Topics in Cognitive Science*, 7(2):217–229, 2015.

[10] D. Kahneman. *Thinking, Fast and Slow.* Farrar, Straus and Giroux, 2011.

[11] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[12] P. D. McCarthy. Graph structure is necessary for information preservation under bounded context. Working paper, 2026.

[13] H. A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118, 1955.

[14] H. A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

[15] O. E. Williamson. Hierarchical control and optimum firm size. *Journal of Political Economy*, 75(2):123–138, 1967.